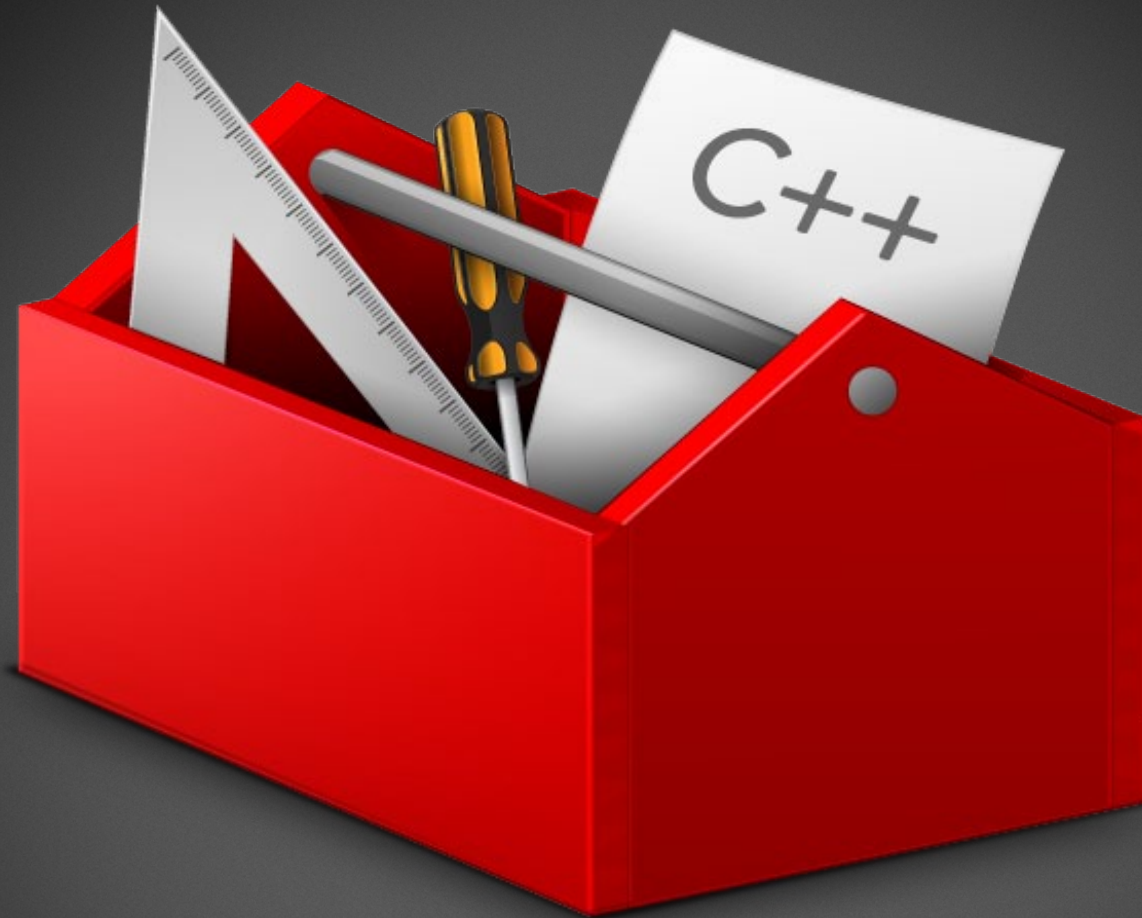


SDK–Videohub Software Developers Kit

Blackmagicdesign 



Mac OS X™

Windows™

August 2010

TABLE OF CONTENTS

1	Introduction	5
1.1	Welcome	5
1.2	Overview	5
1.3	API Design	6
1.3.1	Overview	6
1.3.2	Object Model	6
1.3.3	Object Interfaces	6
1.3.4	Reference Counting	7
1.3.5	Interface Stability	7
1.3.5.1	New Interfaces	7
1.3.5.2	Updated Interfaces	8
1.3.5.3	Deprecated Interfaces	8
1.3.5.4	Removed interfaces	8
1.4	Interface Reference	9
1.4.1	IUnknown Interface	9
1.4.1.1	IUnknown::QueryInterface method	10
1.4.1.2	IUnknown::AddRef method	11
1.4.1.3	IUnknown::Release method	11
2	Videohub API	12
2.1	Using the Videohub API in a project	12
2.2	Accessing Videohub devices	12
2.2.1	Mac OS X	13
2.2.2	Windows	13
2.3	High level interface	14
2.3.1	Videohub Device Discovery	14
2.3.2	Videohub Device Control	15
2.4	Interface Reference	16
2.4.1	IVideohubDiscovery Interface	16
2.4.1.1	IVideohubDiscovery::AddVideohubDiscoveryCallback method	17
2.4.1.2	IVideohubDiscovery::RemoveVideohubDiscoveryCallback method	18
2.4.1.3	IVideohubDiscovery::ConnectTo method	19
2.4.2	IVideohubDiscoveryCallback Interface	20
2.4.2.1	IVideohubDiscoveryCallback::NewVideohubDevice method	21

TABLE OF CONTENTS

2.4.2.2	IVideohubDiscoveryCallback::VideohubDeviceRemoved method	22
2.4.3	IVideohub Interface	23
2.4.3.1	IVideohub::AddVideohubCallback method	24
2.4.3.2	IVideohub::RemoveVideohubCallback method	25
2.4.3.3	IVideohub::GetState method	26
2.4.3.4	IVideohub::SetLabel method	27
2.4.3.5	IVideohub::SetRoute method	28
2.4.3.6	IVideohub::SetLock method	29
2.4.3.7	IVideohub::SetDeviceName method [currently not supported]	30
2.4.4	IVideohubCallback Interface	31
2.4.4.1	IVideohubCallback::StateChanged method	32
2.4.4.2	IVideohubCallback::DeviceDisconnected method	32
2.4.4.3	IVideohubCallback::DeviceReconnected method	32
2.4.4.4	IVideohubCallback::NewDeviceConnected method [currently not supported]	33
2.4.5	IVideohubState Interface	34
2.4.5.1	IVideohubState::GetLabel method	35
2.4.5.2	IVideohubState::GetRoute method	36
2.4.5.3	IVideohubState::GetLock method	37
2.4.5.4	IVideohubState::GetDeviceName method [currently not supported]	38
2.4.5.5	IVideohubState::UpdateState method	39
2.4.6	IVideohubStateChangelterator Interface	40
2.4.6.1	IVideohubStateChangelterator::Next method	41
2.4.6.2	IVideohubStateChangelterator::Reset method	42
2.4.7	IVideohubAttributes Interface	43
2.4.7.1	IVideohubAttributes::GetFlag method	44
2.4.7.2	IVideohubAttributes::GetFloat method	45
2.4.7.3	IVideohubAttributes::GetString method	46
2.4.7.4	IVideohubAttributes::GetInt method	47
2.5	Common Data Types	48
2.5.1	Basic Types	48
2.5.2	Object Types	48
2.5.3	Port Types	49
2.5.4	Lock States	49
2.5.5	Videohub Attribute ID	50
2.5.6	Special Port ID	50

TABLE OF CONTENTS

3	Videohub Ethernet Protocol	52
3.1	Blackmagic Videohub Ethernet Protocol v2.1	52
3.1.1	Summary	52
3.1.2	Protocol Preamble	52
3.1.3	Device information	53
3.1.4	Status updates	57
3.1.5	Requesting changes	58
3.1.6	Requesting a status dump	60
3.1.7	Checking the connection	60
3.2	Videohub RS-422 Protocol	61
3.2.1	General	61
3.2.2	Notifications	62
3.2.3	Global Commands	62
3.2.4	Immediate Commands	63
3.2.5	Salvo Commands	63

SECTION 1 INTRODUCTION

1.1 Welcome

Thanks for downloading the Blackmagic Design Videohub Software Developers Kit.

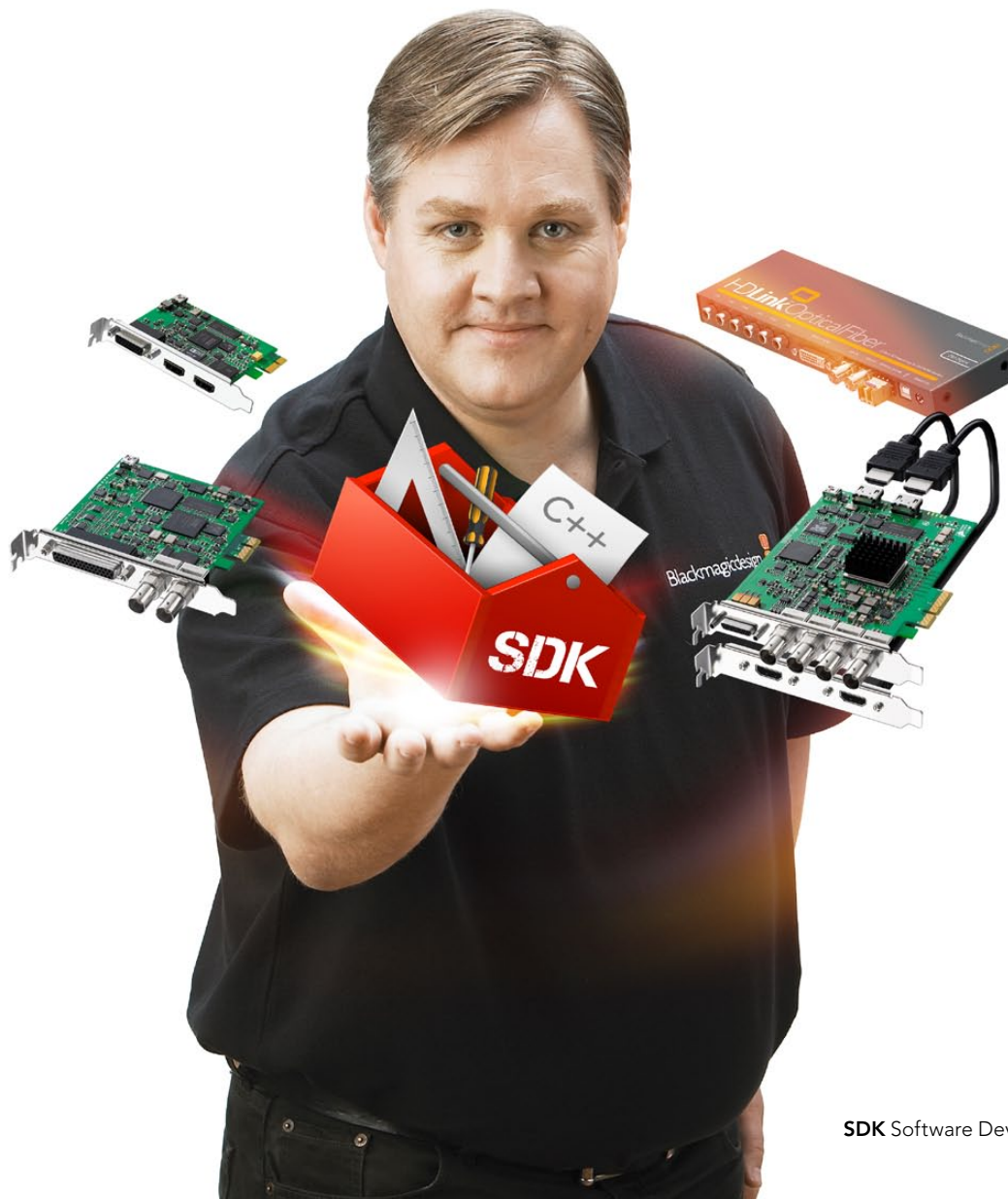
1.2 Overview

The Videohub SDK provides a stable, cross- platform interface to a Blackmagic Design Videohub product line. The SDK provides both low-level control of hardware and high-level interfaces to allow developers to easily perform common tasks.

The SDK consists of a set of interface descriptions & sample applications which demonstrate the use of the basic features of the hardware. The details of the SDK are described in this document.

The SDK supports Microsoft Windows XP, Vista and Mac OS X platforms.

You can download the SDK from www.blackmagic-design.com/support



1.3.1 Overview

The libraries supporting the Blackmagic SDK are shipped as part of the product installers for each supported product line. Applications built against the interfaces shipped in the SDK will dynamically link against the library installed on the end-user's system.

1.3.2 Object Model

The SDK interface is modeled on Microsoft's Component Object Model (COM).

On Microsoft Windows platforms, it is provided as a native COM interface registered with the operating system. On other platforms application code is provided to allow the same COM style interface to be used.

The COM model provides a paradigm for creating flexible and extensible interfaces without imposing much overhead or baggage.

1.3.3 Object Interfaces

The Videohub API provides programmatic access to all current Blackmagic Design Videohub products. "Videohub" is used as a generic term to refer to all supported products and models except where noted.

The API provides high-level interfaces to allow read and write access to port labels, routes and locks.

Functionality within the API is accessed via "object interfaces". Each object in the system may inherit from and be accessed via a number of object interfaces. Typically the developer is able to interact with object interfaces and leave the underlying objects to manage themselves.

Each object interface class has a Globally Unique ID (GUID) called an “Interface ID”. On platforms with native COM support, an IID may be used to obtain a handle to an exported interface object from the OS, which is effectively an entry point to an installed API.

Each interface may have related interfaces that are accessed by providing an IID to an existing object interface (see `IUnknown::QueryInterface`). This mechanism allows new interfaces to be added to the API without breaking API or ABI compatibility.

1.3.4 Reference Counting

The API uses reference counting to manage the life cycle of object interfaces.

The developer may need to add or remove references on object interfaces (see `IUnknown::AddRef` and `IUnknown::Release`) to influence their life cycle as appropriate in the application.

1.3.5 Interface Stability

The SDK provides a set of stable interfaces for accessing Blackmagic Design hardware. Whilst the published interfaces will remain stable, developers need to be aware of some issues they may encounter as new products, features and interfaces become available.

1.3.5.1 New Interfaces

Major pieces of new functionality may be added to the SDK as a whole new object interface. Already released applications will not be affected by the additional functionality. Developers making use of the new functionality should be sure to check the return of `CoCreateInstance` and/or `QueryInterface` as these interfaces will not be available on users systems which are running an older release of the Blackmagic drivers.

Developers can choose to either reduce the functionality of their application when an interface is not available, or to notify the user that they must install a later version of the Blackmagic drivers.

1.3.5.2 Updated Interfaces

As new functionality is added to the SDK, some existing interfaces may need to be modified or extended. To maintain compatibility with released software, the original interface will be deprecated but will remain available and maintain its unique identifier (IID). The replacement interface will have a new identifier and remain as similar to the original as possible.

1.3.5.3 Deprecated Interfaces

Interfaces which have been replaced with an updated version, or are no longer recommended for use are “deprecated”. Deprecated interfaces are moved out of the main interface description files into an interface description file named according to the release in which the interface was deprecated. Deprecated interfaces are also renamed with a suffix indicating the release prior to the one in which they were deprecated..

It is recommended that developers update their applications to use the most recent SDK interfaces when they release a new version of their applications. As an interim measure, developers may include the deprecated interface descriptions, and updating the names of the interfaces in their application to access the original interface functionality.

1.3.5.4 Removed interfaces

Interfaces that have been deprecated for some time may eventually be removed in a major driver update if they become impractical to support.

1.4 Interface Reference

Every object interface subclasses the **IUnknown** interface.

1.4.1 IUnknown Interface

Each API interface is a subclass of the standard COM base class – **IUnknown**. The **IUnknown** object interface provides reference counting and the ability to look up related interfaces by interface ID. The interface ID mechanism allows interfaces to be added to the API without impacting existing applications.

Public Member Functions	
Method	Description
QueryInterface	Provides access to supported child interfaces of the object.
AddRef	Increments the reference count of the object.
Release	Decrements the reference count of the object. When the final reference is removed, the object is freed.

1.4.1.1 IUnknown::QueryInterface method

The **QueryInterface** method looks up a related interface of an object interface.

Syntax

```
HRESULT QueryInterface(REFIID id, void **outputInterface);
```

Parameters

Name	Direction	Description
id	in	Interface ID of interface to lookup
outputInterface	out	New object interface or NULL on failure.

Return Values

Value	Description
E_NOINTERFACE	Interface was not found
S_OK	Success

1.4.1.2 IUnknown::AddRef method

The **AddRef** method increments the reference count for an object interface.

Syntax

```
ULONG          AddRef();
```

Parameters

none.

Return Values

Value	Description
Count	New reference count – for debug purposes only.

1.4.1.3 IUnknown::Release method

The **Release** method decrements the reference count for an object interface. When the last reference is removed from an object, the object will be destroyed.

Syntax

```
ULONG          Release();
```

Parameters

none.

Return Values

Value	Description
Count	New reference count – for debug purposes only.

SECTION 2 Videohub API

2.1 Using the Videohub API in a project

The supplied sample applications provide examples of how to include the Videohub API in a project on each supported platform.

To use the Videohub API in your project, one or more files need to be included:

Windows	Videohub X.Y\Win\Include\VideohubAPI.idl
	Videohub X.Y\Win\Include\VideohubAPIDispatch.cpp (optional)
	Videohub X.Y\Win\Include\VideohubAPIDispatch.h (optional)
Mac OS X	Videohub X.Y/Mac/Include/VideohubAPI.h
	Videohub X.Y/Mac/Include/VideohubAPIDispatch.cpp

2.2 Accessing Videohub devices

Most Videohub API object interfaces are accessed via the **IVideohubDiscovery** object. How a reference to an **IVideohubDiscovery** is obtained varies between platforms depending on their level of support for COM:

SECTION 2 Videohub API

2.2.1 Mac OS X

On platforms without native COM support, a C entry point is provided to access an **IVideohubDiscovery** object:

```
IVideohubDiscovery * videohubDiscovery = CreateVideohubDiscoveryInstance(NULL);
```

On success, videohubDiscovery will point to a new **IVideohubDiscovery** object interface, otherwise it will be set to NULL.

The application may pass the full path to the API library to **CreateVideohubDiscoveryInstance** or pass NULL to load the library from the default location.

2.2.2 Windows

An **IVideohubDiscovery** object interface should be obtained from COM using **CoCreateInstance**:

```
IVideohubDiscovery *videohubDiscovery = NULL;
```

```
CoCreateInstance(CLSID_CVideohubDiscovery, NULL, CLSCTX_ALL,  
IID_IVideohubDiscovery, (void*)&videohubDiscovery);
```

On success, **CoCreateInstance** returns an HRESULT of S_OK and videohubDiscovery points to a new **IVideohubDiscovery** object interface.

Windows applications may use **CreateVideohubDiscoveryInstance** as an alternative to locating the API library using the COM registry. To use the entry point, the VideohubAPIDispatch header and source file should be included, and **CreateVideohubDiscoveryInstance** called in the same way as for Mac (see above).

SECTION 2 Videohub API

2.3 High level interface

The Videohub API provides a framework for discovering and controlling Videohub devices. This section provides an overview of how to use these interfaces.

2.3.1 Videohub Device Discovery

The Videohub API relies on Bonjour to automate Videohub device discovery. On Windows platforms, the Apple Bonjour For Windows must be installed in order to use the Videohub device discovery feature. Apple Bonjour For Windows can be downloaded from:

http://support.apple.com/downloads/Bonjour_for_Windows

Provided the Bonjour library is available, an application should perform the following steps to discover Videohub devices:

IVideohubDiscovery::AddVideohubDiscoveryCallback(discovery callback)

Calls will be made to the provided callback object with a list of new and removed Videohub device addresses.

2.3.2 Videohub Device Control

Given a Videohub device address, an application should perform the following steps to obtain an `IVideohub` object which can then be used to control the device and access its state:

- **`IVideohubDiscovery::ConnectTo`** (device address, timeout, `IVideohub` object)
- **`IVideohub::AddVideohubCallback`** (videohub callback)

Calls will be made to the videohub callback object whenever the Videohub device is disconnected, reconnected, or when its state changes.

The following method can be used to access the device's current state:

- **`IVideohub::GetState`** (videohub state object)

When the state of the device changes, the application should call **`IVideohubState::UpdateState`** to retrieve the current state. This method returns an **`IVideohubStateIterator`** object which enumerates the changes since the last state was retrieved.

The following methods can be used to change the state of the device:

- **`IVideohub::SetLabel`** (port type, port index, new label)
- **`IVideohub::SetRoute`** (output port type, output port index, input port index)
- **`IVideohub::SetLock`** (port type, port index, lock state)

2.4 Interface Reference

2.4.1 IVideohubDiscovery Interface

The **AddVideohubDiscoveryCallback** method registers a Videohub discovery callback object which will be called when a new Videohub device becomes available, or an existing one is removed.

A reference to an **IVideohubDiscovery** object interface may be obtained from **CoCreateInstance** on platforms with native COM support or from **CreateIVideohubDiscoveryInstance** on other platforms.

Related Interfaces

Interface	Interface ID	Description
IVideohub	IID_IVideohub	an IVideohub object can be obtained from IVideohubDiscovery::ConnectTo
IVideohub DiscoveryCallback	IID_IVideohub DiscoveryCallback	An IVideohubDiscoveryCallback object interface may be registered with IVideohubDiscovery::AddVideohubDiscoveryCallback

Public Member Functions

Method	Description
ConnectTo	Returns an IVideohub object interface corresponding to a Videohub device.
AddVideohubDiscoveryCallback	Registers a Videohub discovery callback object.
RemoveVideohubDiscoveryCallback	Unregisters a Videohub discovery callback object.

2.4.1.1 **IVideohubDiscovery::AddVideohubDiscoveryCallback** method

The **AddVideohubDiscoveryCallback** method registers a Videohub discovery callback object which will be informed when a new Videohub device becomes available, or an existing one is removed. Videohub devices are referred to using their device address.

Note: On Windows platforms, Apple's Bonjour For Windows must be installed in order to make use of the automated Videohub discovery feature. It can be downloaded from http://support.apple.com/downloads/Bonjour_for_Windows

Syntax

```
HRESULT          AddVideohubDiscoveryCallback
                  (IVideohubDiscoveryCallback *discoveryCallback)
```

Parameters

Name	Direction	Description
discoveryCallback	in	The IVideohubDiscoveryCallback object to be registered.

Return Values

Value	Description
E_POINTER	The given callback object is NULL
E_NOINTERFACE	The Bonjour library is not installed (Windows only)
S_OK	Success

2.4.1.2 **IVideohubDiscovery::RemoveVideohubDiscoveryCallback** method

The **RemoveVideohubDiscoveryCallback** method unregisters a previously-registered Videohub discovery callback object.

Syntax

```
HRESULT RemoveVideohubDiscoveryCallback
(IVideohubDiscoveryCallback *discoveryCallback)
```

Parameters

Name	Direction	Description
discoveryCallback	in	The IVideohubDiscoveryCallback object to be unregistered.

Return Values

Value	Description
E_POINTER	The given callback object is NULL
E_INVALIDARG	The given callback object has not been previously registered
E_NOINTERFACE	The Bonjour library is not installed (Windows only)
S_OK	Success

2.4.1.3 IVideohubDiscovery::ConnectTo method

The **ConnectTo** method connects to a Videohub device given its device address, as passed to a Videohub discovery callback object.

The device address may be in the form of an IP address (eg: 1.2.3.4) or an IP address and a port number (eg: 1.2.3.4:5678)

The second parameter specifies a connection timeout value. If a negative value is passed, **ConnectTo** will block indefinitely until a connection to the device can be made. If positive, the value represents a timeout in milliseconds and **ConnectTo** will return as soon as a connection can be made, or when the timeout expires.

Syntax

```
HRESULT          ConnectTo (string deviceAddress,  int32_t connectionTimeout, IVideohub **videoHubDevice)
```

Parameters

Name	Direction	Description
deviceAddress	in	The address of the Videohub device to connect to, as passed to a Videohub discovery callback object
connectionTimeout	in	A connection timeout value in milliseconds
videohubDevice	out	A IVideohub object interface

Return Values

Value	Description
E_FAIL	A connection to the Videohub device at the given address could not be made
S_OK	Success
E_NOTIMPL	The Videohub server is incompatible with this API

SECTION 2 Videohub API

2.4.2 **IVideohubDiscoveryCallback** Interface

The **IVideohubDiscoveryCallback** object interface is a callback class which is called whenever a new Videohub device becomes available, or an existing one is removed. Videohub devices are referred to using their device address.

An object with an **IVideohubDiscoveryCallback** interface may be registered as a callback with the **IVideohubDiscovery** interface.

Related Interfaces

Interface	Interface ID	Description
IVideohubDiscovery	IID_IVideohubDiscovery	An IVideohubDiscoveryCallback object can be registered with IVideohubDiscovery::AddVideohubDiscoveryCallback

Public Member Functions

Method	Description
NewVideohubDevice	DeviceDiscovered
VideohubDeviceRemoved	DeviceGone

2.4.2.1 **VideohubDiscoveryCallback::NewVideohubDevice** method

The **NewVideohubDevice** method is called when a new Videohub device becomes available. This method is abstract in the base interface and must be implemented by the application developer. The result parameter (required by COM) is ignored by the caller.

Syntax

HRESULT **NewVideohubDevice** (string deviceAddress)

Parameters

Name	Direction	Description
deviceAddress	in	The address of the new Videohub device. This allocated string must be freed by the caller when no longer required.
displayableName	in	A name suitable for displaying in a user interface. This allocated string must be freed by the caller when no longer required.

2.4.2.2 VideohubDiscoveryCallback::VideohubDeviceRemoved method

The **VideohubDeviceRemoved** method is called when an existing Videohub device is removed. This method is abstract in the base interface and must be implemented by the application developer. The result parameter (required by COM) is ignored by the caller.

Syntax

HRESULT VideohubDeviceRemoved (string deviceAddress)

Parameters

Name	Direction	Description
deviceAddress	in	The address of the Videohub device. This dynamically-allocated string must be freed by the caller when no longer required.
displayableName	in	A name suitable for displaying in a user interface. This dynamically-allocated string must be freed by the caller when no longer required.

2.4.3 IVideohub Interface

The **IVideohub** object interface is used to access or change the current state of a Videohub.

A reference to an **IVideohub** object interface may be obtained from **IVideohubDiscovery**.

IVideohub can be queried to obtain the related **IVideohubAttributes** interface.

Related Interfaces

Interface	Interface ID	Description
IVideohubAttributes	IID_IVideohubAttributes	An IVideohubAttributes object can be obtained from IVideohub using QueryInterface .
IVideohubCallback	IID_IVideohubCallback	An IVideohubCallback object interface may be registered with IVideohub::AddVideohubCallback .
IVideohubState	IID_IVideohubState	An IVideohubState object can be obtained from IVideohub::GetState .

Public Member Functions

Method	Description
GetState	Returns the current state of the Videohub device.
SetLabel	Applies a new label to a port on the Videohub device
SetRoute	Changes the routing of a port on the Videohub device
SetLock	Acquires/Releases the lock on a port
AddVideohubCallback	Registers a Videohub callback object.
RemoveVideohubCallback	Unregisters a Videohub callback object.

SECTION 2 Videohub API

2.4.3.1 **IVideohub::AddVideohubCallback** method

The **AddVideohubCallback** method registers a Videohub callback object which will be informed when the state of the Videohub device changes, the device is disconnected or reconnected.

Syntax

HRESULT AddVideohubCallback (IVideohubCallback *videohubCallback)

Parameters

Name	Direction	Description
videohubCallback	in	The IVideohubCallback object to be registered.

Return Values

Value	Description
E_INVALIDARG	The given callback object is NULL
S_OK	Success

2.4.3.2 **IVideohub::RemoveVideohubCallback** method

The **RemoveVideohubCallback** method unregisters a previously-registered Videohub callback object.

Syntax

HRESULT RemoveVideohubCallback (IVideohubCallback *videohubCallback)

Parameters

Name	Direction	Description
videohubCallback	in	The IVideohubCallback object to be unregistered.

Return Values

Value	Description
E_INVALIDARG	The given callback object is NULL or has not been registered previously
S_OK	Success

SECTION 2 Videohub API

2.4.3.3 **IVideohub::GetState** method

The **GetState** method returns an **IVideohubState** object interface which encapsulates the state of the Videohub device at the time the call is made.

Syntax

HRESULT GetState (IVideohubState **currentState)

Parameters

Name	Direction	Description
currentState	out	The IVideohubState object representing the current state of the Videohub device

Return Values

Value	Description
E_FAIL	The Videohub device is currently disconnected
S_OK	Success

2.4.3.4 IVideohub::SetLabel method

The **SetLabel** method assigns a new label to a port on the Videohub device.

This method sends a command to a Videohub device, and the returned value indicates whether the command was successfully sent. If the change was successfully applied, a state update event will be received asynchronously.

Syntax

HRESULT SetLabel (BMDVideohubPortType portType, uint32_t portIndex, string newLabel)

Parameters

Name	Direction	Description
portType	in	The port type
portIndex	in	The index of the port
newLabel	in	The new label

Return Values

Value	Description
E_INVALIDARG	One of the arguments is invalid
E_FAIL	The Videohub device is currently disconnected
S_OK	Success

2.4.3.5 IVideohub::SetRoute method

The **SetRoute** method sets up a new route between a video input port and either a video output or monitoring output.

This method sends a command to a Videohub device, and the returned value indicates whether the command was successfully sent. If the change was successfully applied, a state update event will be received asynchronously.

Syntax

HRESULT SetRoute (BMDVideohubPortType outputPortType, uint32_t outputPortIndex, uint32_t inputPortIndex)

Parameters

Name	Direction	Description
outputPortType	in	The output port type
outputPortIndex	in	The output port index
inputPortIndex	in	The input port index

Return Values

Value	Description
E_INVALIDARG	One of the arguments is invalid
E_FAIL	The Videohub device is currently disconnected
S_OK	Success

2.4.3.6 IVideohub::SetLock method

The **SetLock** method either acquires or releases a lock on a video output, monitoring output or serial port.

This method sends a command to a Videohub device, and the returned value indicates whether the command was successfully sent. If the change was successfully applied, a state update event will be received asynchronously.

Syntax

HRESULT SetLock (BMDVideohubPortType portType, uint32_t portIndex, BMDVideohubLockState newState)

Parameters

Name	Direction	Description
portType	in	The port type
portIndex	in	The port index
newState	in	The lock state

Return Values

Value	Description
E_INVALIDARG	One of the arguments is invalid
E_FAIL	The Videohub device is currently disconnected
S_OK	Success

2.4.3.7 **IVideohub::SetDeviceName** method [currently not supported]

The **SetDeviceName** method sets the name of the Videohub device.

This method sends a command to a Videohub device, and the returned value indicates whether the command was successfully sent. If the change was successfully applied, a state update event will be received asynchronously.

Syntax

HRESULT SetDeviceName (string newName)

Parameters

Name	Direction	Description
newName	in	The device's new name

Return Values

Value	Description
E_FAIL	The Videohub device is currently disconnected
S_OK	Success

SECTION 2 Videohub API

2.4.4 **IVideohubCallback** Interface

The **IVideohubCallback** object interface is a callback class which is called whenever a Videohub device is disconnected, reconnected or when its state changes.

An object with an **IVideohubCallback** interface may be registered as a callback with the **IVideohub** interface.

Related Interfaces

Interface	Interface ID	Description
IVideohub	IID_IVideohub	An IVideohubCallback object can be registered with IVideohub::AddVideohubCallback

Public Member Functions

Method	Description
StateChanged	Called when the state of the Videohub device has changed
DeviceDisconnected	Called when the Videohub device is disconnected
DeviceReconnected	Called when the same Videohub device is reconnected
NewDeviceConnected	Called when a different Videohub device is connected

SECTION 2 Videohub API

2.4.4.1 IVideohubCallback::StateChanged method

The **StateChanged** method is called when the state of the Videohub device has changed. This method is abstract in the base interface and must be implemented by the application developer. The result parameter (required by COM) is ignored by the caller.

Syntax

HRESULT StateChanged ()

2.4.4.2 IVideohubCallback::DeviceDisconnected method

The **DeviceDisconnected** method is called when the Videohub device is disconnected. This method is abstract in the base interface and must be implemented by the application developer. The result parameter (required by COM) is ignored by the caller.

Syntax

HRESULT DeviceDisconnected ()

2.4.4.3 IVideohubCallback::DeviceReconnected method

The **DeviceReconnected** method is called when the Videohub device is reconnected. This method is abstract in the base interface and must be implemented by the application developer. The result parameter (required by COM) is ignored by the caller.

Syntax

HRESULT DeviceReconnected ()

SECTION 2 Videohub API

2.4.4.4 **IVideohubCallback::NewDeviceConnected** method [currently not supported]

The **NewDeviceConnected** method is called when a different Videohub device is connected. This method is abstract in the base interface and must be implemented by the application developer. The result parameter (required by COM) is ignored by the caller.

Syntax

HRESULT NewDeviceConnected ()

2.4.5 IVideohubState Interface

The **IVideohubState** object interface encapsulates the state of a Videohub at a given time.

A reference to an **IVideohubState** object interface may be obtained from **IVideohub**.

Related Interfaces

Interface	Interface ID	Description
IVideohub	IID_IVideohub	An IVideohubState object interface can be obtained from IVideohub using GetState .
IVideohubStateChangeIterator	IID_IVideohubStateChangeIterator	An IVideohubStateChangeIterator object interface can be obtained from IVideohubState using UpdateState

Public Member Functions

Method	Description
GetLabel	Returns the label for a given port
GetRoute	Returns the route of a port
GetLock	Returns the lock state on a port
GetDeviceName	Returns the device's name
UpdateState	Updates this object with the current state

2.4.5.1 IVideohubState::GetLabel method

The **GetLabel** method returns the label for a given port.

Syntax

HRESULT GetLabel (BMDVideohubPortType portType, uint32_t portIndex, string* label)

Parameters

Name	Direction	Description
portType	in	The port type
portIndex	in	The port index
label	out	The label. This allocated string must be freed by the caller when no longer required.

Return Values

Value	Description
E_INVALIDARG	One of the arguments is invalid
S_OK	Success

2.4.5.2 IVideohubState::GetRoute method

The **GetRoute** method returns the routing for a given video output or monitoring output.

Syntax

HRESULT GetRoute (BMDVideohubPortType portType, uint32_t outputPortIndex, uint32_t *inputPortIndex)

Parameters

Name	Direction	Description
portType	in	The output port type
outputPortIndex	in	The output port index
inputPortIndex	out	The input port index

Return Values

Value	Description
E_INVALIDARG	One of the arguments is invalid
S_OK	Success

2.4.5.3 IVideohubState::GetLock method

The **GetLock** method returns the lock state for a given port.

Syntax

HRESULT GetLock (BMDVideohubPortType portType, uint32_t portIndex, BMDVideohubLockState *lock)

Parameters

Name	Direction	Description
portType	in	The port type
portIndex	in	The port index
lock	out	The lock state

Return Values

Value	Description
E_INVALIDARG	One of the arguments is invalid
S_OK	Success

2.4.5.4 **IvideohubState::GetDeviceName** method [currently not supported]

The **GetDeviceName** method returns the name of the Videohub device.

Syntax

HRESULT GetDeviceName (string *name)

Parameters

Name	Direction	Description
name	out	The device's name. This allocated string must be freed by the caller when no longer required.

Return Values

Value	Description
Value	Description
S_OK	Success

2.4.5.5 IVideohubState::UpdateState method

The **UpdateState** method updates the state of this **IVideohubState** object interface and returns an **IVideohubStateChangeIterator** object interface which can be used to iterate over the items whose value has changed.

Syntax

HRESULT UpdateState (IVideohubStateChangeIterator **changeIterator)

Parameters

Name	Direction	Description
changeIterator	out	An iterator over the items whose value has changed as a result of the update.

Return Values

Value	Description
E_INVALIDARG	This IVideohubState object can not be updated because the Videohub device which generated it is no longer connected. Instead, request a new state object using IVideohub::GetState
E_FAIL	The Videohub device is currently disconnected
S_FALSE	No changes have occurred
S_OK	Success

2.4.6 IVideohubStateChangelterator Interface

The **IVideohubStateChangelterator** object interface can be used to iterate over the changes made to an **IVideohubState** object interface as a result of an update.

A reference to an **IVideohubStateChangelterator** object interface may be obtained from **IVideohubState**.

Related Interfaces

Interface	Interface ID	Description
IVideohubState	IID_IVideohubState	An IVideohubStateChangelterator object interface can be obtained from IVideohubState using UpdateState .

Public Member Functions	
Method	Description
Next	Returns details about the next item whose values have been updated
Reset	Resets this iterator

2.4.6.1 IVideohubStateChangelterator::Next method

The **Next** method returns the details about the next item whose values have been updated.

Syntax

HRESULT **Next** (BMDVideohubObjectType *object, BMDVideohubPortType *portType, uint32_t *portIndex)

Parameters

Name	Direction	Description
object	out	The object type whose value has been updated
portType	in	The port type
portIndex	in	The port index

Return Values

Value	Description
S_OK	Success
S_FALSE	There are no more changes. The iterator may be reset using the Reset method if desired

SECTION 2 Videohub API

2.4.6.2 **IVideohubStateChangelterator::Reset** method

The **Reset** method resets this iterator. Subsequent calls to **Next** will iterate over the entire set of changes starting with first item.

Syntax

HRESULT **Reset** ()

Return Values

Value	Description
S_OK	Success

2.4.7 IVideohubAttributes Interface

The **IVideohubAttributes** object interface provides details about the capabilities of a Videohub device. The detail types that are available for various capabilities are: flag, int, float, and string. The Videohub Attribute ID section lists the hardware capabilities and associated attributes identifiers that can be queried using this object interface. An **IVideohubAttributes** object interface can be obtained from the **IVideohub** interface using **QueryInterface**.

Public Member Functions	
Method	Description
GetFlag	Gets a boolean flag corresponding to a BMDVideohubAttributeID
GetFloat	Gets a float corresponding to a BMDVideohubAttributeID
GetString	Gets a string corresponding to a BMDVideohubAttributeID
GetInt	Gets an int corresponding to a BMDVideohubAttributeID

2.4.7.1 **VideohubAttributes::GetFlag** method

The **GetFlag** method returns a boolean flag associated with a given **BMDVideohubAttributeID**. (See Videohub Attribute ID section for a list of attribute identifiers).

Syntax

HRESULT GetFlag (BMDVideohubAttributeID cfgID, BMDbool *value)

Parameters

Name	Direction	Description
cfgID	in	The attribute identifier
value	out	The boolean flag

Return Values

Value	Description
E_FAIL	Failure
E_INVALIDARG	No boolean flag for the given attribute id
S_OK	Success

2.4.7.2 IVideohubAttributes::GetFloat method

The **GetFloat** method returns a float value associated with a given **BMDVideohubAttributeID**. (See Videohub Attribute ID section for a list of attribute identifiers).

Syntax

HRESULT GetFloat (BMDVideohubAttributeID cfgID, double *value)

Parameters

Name	Direction	Description
cfgID	in	The attribute identifier
value	out	The float value

Return Values

Value	Description
E_FAIL	Failure
E_INVALIDARG	No float value for the given attribute id
S_OK	Success

2.4.7.3 IVideohubAttributes::GetString method

The **GetString** method returns a string associated with a given **BMDVideohubAttributeID**. (See Videohub Attribute ID section for a list of attribute identifiers).

Syntax

HRESULT GetString (BMDVideohubAttributeID cfgID, string *value)

Parameters

Name	Direction	Description
cfgID	in	The attribute identifier
value	out	The string. This allocated string must be freed by the caller when no longer required.

Return Values

Value	Description
E_FAIL	Failure
E_INVALIDARG	No string for the given attribute id
S_OK	Success

2.4.7.4 IVideohubAttributes::GetInt method

The **GetInt** method returns an `int64_t` associated with a given **BMDVideohubAttributeID**. (See Videohub Attribute ID section for a list of attribute identifiers).

Syntax

```
HRESULT          GetInt (BMDVideohubAttributeID cfgID, int64_t *value)
```

Parameters

Name	Direction	Description
cfgID	in	The attribute identifier
value	out	The integer value

Return Values

Value	Description
E_FAIL	Failure
E_INVALIDARG	No integer value for the given attribute id
S_OK	Success

2.5 Common Data Types

2.5.1 Basic Types

boolean

boolean is represented differently on each platform by using its system type:

Windows	BOOL
Mac OS X	bool

Strings

Strings are represented differently on each platform, using the most appropriate system type:

Windows	BSTR
Mac OS X	CFStringRef

Strings returned by methods in this API must be released by the caller when no longer needed.

2.5.2 Object Types

BMDVideohubObjectType enumerates the different Videohub object types:

Object type	Description
BMDVideohubLabel	A label associated with a port
BMDVideohubLock	A lock associated with a port
BMDVideohubRoute	A route between an input and output port

2.5.3 Port Types

BMDVideohubPortType enumerates the different types of port found on Videohub devices:

Port type	Description
BMDVideohubSerialPort	A serial port
BMDVideohubVideoInputPort	A video input port
BMDVideohubVideoOutputPort	A video output port
BMDVideohubVideoMonitoringOutputPort	A video monitoring output port

2.5.4 Lock States

BMDVideohubLockState enumerates the states of a lock:

Port type	Description
BMDVideohubLockOwned	The application currently owns the lock.
BMDVideohubLockUnlocked	No application currently owns the lock.
BMDVideohubLockTaken	The lock is currently owned by another application.

SECTION 2 Videohub API

2.5.5 Videohub Attribute ID

BMDVideohubAttributeID enumerates a set of attributes of a Videohub device that can be queried using an **IVideohubAttributes** object interface:

Port type	Type	Description
BMDVideohubNumInputPorts	Int	The number of input ports on the device
BMDVideohubNumOutputPorts	Int	The number of output ports on the device
BMDVideohubNumMonitoringOutputPorts	Int	The number of monitoring output ports on the device
BMDVideohubNumSerialPorts	Int	The number of serial ports on the device
BMDVideohubUniqueID	String	The unique identifier of the device. [currently not supported]

2.5.6 Special Port ID

BMDVideohubSpecialPortIndex enumerates special port index values:

Name	Description
BMDVideohubPortDisconnected	When this value is reported by the API, it indicates the corresponding port is disconnected. This value can only be given as an argument to IVideohub::SetRoute

SECTION 2 Videohub API

2.5.5 Videohub Attribute ID

BMDVideohubAttributeID enumerates a set of attributes of a Videohub device that can be queried using an **IVideohubAttributes** object interface:

Port type	Type	Description
BMDVideohubNumInputPorts	Int	The number of input ports on the device
BMDVideohubNumOutputPorts	Int	The number of output ports on the device
BMDVideohubNumMonitoringOutputPorts	Int	The number of monitoring output ports on the device
BMDVideohubNumSerialPorts	Int	The number of serial ports on the device
BMDVideohubUniqueID	String	The unique identifier of the device. [currently not supported]

2.5.6 Special Port ID

BMDVideohubSpecialPortIndex enumerates special port index values:

Name	Description
BMDVideohubPortDisconnected	When this value is reported by the API, it indicates the corresponding port is disconnected. This value can only be given as an argument to IVideohub::SetRoute

SECTION 3 Videohub Ethernet Protocol

3.1 Blackmagic Videohub Ethernet Protocol v2.1

3.1.1 Summary

The Blackmagic Videohub Ethernet Protocol is a text based protocol that is accessed by connecting to TCP port 9990 on a Videohub Server. Integrated Videohub Servers and Videohub Server computers are supported by the protocol. Each line in the protocol is terminated by a newline character.

Upon connection, the Videohub Server sends a complete dump of the state of the device. After the initial status dump, status updates are sent every time the Videohub status changes.

The Videohub Server sends information in blocks which each have an identifying header in all-caps, followed by a full-colon. A block spans multiple lines and is terminated by a blank line.

To be resilient to future protocol changes, clients should ignore blocks they do not recognize, up to the trailing blank line. Within existing blocks, clients should ignore lines they do not recognize.

3.1.2 Protocol Preamble

The first block sent by the Videohub Server is always the protocol preamble:

```
PROTOCOL PREAMBLE:↵
Version: 2.1 ↵
↵
```

To be resilient to future protocol changes, clients should ignore blocks they do not recognize, up to the trailing blank line. Within existing blocks, clients should ignore lines they do not recognize.

Legend

↵ carriage return
... and so on

Version 2.1 of the Blackmagic Videohub Ethernet Protocol was released with Videohub 4.5.1 software.

SECTION 3 Videohub Ethernet Protocol

3.1.3 Device information

The next block contains general information about the connected Videohub device.

If a device is connected, the Videohub Server will report the attributes of the Videohub:

```
VIDEOHUB_DEVICE:↵
Device present: true↵
Model name: Blackmagic Smart Videohub↵
Video inputs: 16↵
Video processing units: 0↵
Video outputs: 16↵
Video monitoring outputs: 0↵
Serial ports: 0↵
↵
```

This example is for the Smart Videohub which is a 16x16 router.

If the Videohub Server has no device connected, the block will simply be:

```
VIDEOHUB_DEVICE:↵
Device present: false↵
↵
```

If a device is present, but has an incompatible firmware, the status reported will be:

```
VIDEOHUB_DEVICE:↵
Device present: needs_update↵
↵
```

In both situations, no further information will be sent, unless the situation is rectified.

If the Videohub Server detects a new Videohub attached, it will resend all blocks except the protocol preamble to indicate the device has changed, and allow the client to update its cache of server state.

SECTION 3 Videohub Ethernet Protocol

The next four blocks enumerate the labels assigned to the input, output, monitoring and serial ports.

Note: Videohubs which do not have monitoring or serial ports do not send the corresponding blocks.

```
INPUT LABELS:↵
0 VTR 11 VTR 2↵
...
↵
OUTPUT LABELS:↵
0 Output feed 1↵
1 Output feed 2↵
...
↵
MONITORING OUTPUT LABELS:↵
0 Monitor feed 1↵
1 Monitor feed 2↵
...
↵
SERIAL PORT LABELS:↵
0 Deck 1↵
1 Deck 2↵
↵
```

Note: Ports are always numbered starting at zero in the protocol.

SECTION 3 Videohub Ethernet Protocol

The next three blocks describe the routing of the output, monitoring and serial ports.

VIDEO OUTPUT ROUTING: ↵

0 5↵

1 3↵

...

↵

VIDEO MONITORING OUTPUT ROUTING: ↵

0 7↵

1 8↵

...

↵

SERIAL PORT ROUTING: ↵

0 12↵

1 11↵

...

↵

Videohubs with processing units (only the Workgroup Videohub) send an extra routing block:

PROCESSING UNIT ROUTING: ↵

0 5↵

1 3↵

...

↵

Videohubs with frame buffers (only the Workgroup Videohub) send two extra blocks:

FRAME LABELS: ↵

0 Frame one↵

1 Frame two↵

...

↵

SECTION 3 Videohub Ethernet Protocol

FRAME BUFFER ROUTING:↵

0 7↵

1 8↵

...

↵

The next three blocks describe the locking status of the output, monitoring and serial ports. Each port has a lock status of "O" for ports which are owned by the current client (ie locked from the same IP address), "L" for ports which are locked from a different client, or "U" for unlocked. Note that Videohubs which do not have monitoring ports or serial ports do not send the corresponding blocks.

VIDEO OUTPUT LOCKS:↵

0 U↵

1 U↵

...

↵

MONITORING OUTPUT LOCKS:↵

0 U↵

1 U↵

...

↵

SERIAL PORT LOCKS:↵

0 U↵

1 U↵

...

↵

SECTION 3 Videohub Ethernet Protocol

Videohubs with processing units (only the Workgroup Videohub) send an extra lock block:

```
PROCESSING UNIT LOCKS:↵
0 U↵
1 U↵
...
↵
```

Videohubs with frame buffers (only the Workgroup Videohub) send an extra lock blocks:

```
FRAME BUFFER LOCKS:↵
0 U↵
1 U↵
...
↵
```

3.1.4 Status updates

When any route or label is changed on the Videohub Server by any client, the Videohub Server resends the applicable status block, containing **only** the items which have changed. For example, if serial port 6 has been unlocked, the following block will be sent:

```
SERIAL PORT LOCKS:↵
5 U↵
↵
```

If multiple items are changed, multiple items may be present in the update:

```
OUTPUT LABELS:↵
7 New output 8 label↵
10 New output 11 label↵
↵
```

SECTION 3 Videohub Ethernet Protocol

3.1.5 Requesting changes

To update a label, lock or route, the client should send a block of the same form the Videohub Server sends when its status changes. For example, to change the route of output port 8 to input port 3, the client should send the following block:

```
VIDEO OUTPUT ROUTING:↵  
7 2↵  
↵
```

The block must be terminated by a blank line. On receipt of a blank line, the Videohub Server will either acknowledge the request by responding:

```
ACK↵  
↵
```

or indicate that the response was not understood or able to be actioned by responding:

```
NAK↵  
↵
```

After a positive response, the client should expect to see a status update from the Videohub Server showing the status change. This is likely to be the same as the command that was sent, but if other changes were made simultaneously by other clients, there may be more updates in the block, or more blocks. Simultaneous updates could cancel each other out, leading to a response that is different to that expected.

SECTION 3 Videohub Ethernet Protocol

In the absence of simultaneous updates, the dialog expected for a simple label change is as follows:

```
OUTPUT LABELS:↵
6 new output label seven↵
↵
ACK↵
↵
OUTPUT LABELS:↵
6 new output label seven↵
↵
```

The asynchronous nature of the responses means that a client should never rely on the desired update actually occurring and must simply watch for status updates from the Videohub Server and use only these to update its local representation of the server state.

To lock a port, send an update to the port with the character “O” indicating that you wish to lock the port for example:

```
SERIAL PORT LOCKS:↵
7 O↵
↵
ACK↵
↵
SERIAL PORT LOCKS:↵
7 O↵
↵
```

SECTION 3 Videohub Ethernet Protocol

3.1.6 Requesting a status dump

The client may request that the Videohub Server resend the complete state of any status block by sending the header of the block, followed by a blank line. In the following example, the client requests the Videohub Server resend the output routing:

```
OUTPUT LABELS:↵
↵
ACK↵
↵
OUTPUT LABELS:↵
0 output label 1↵
1 output label 2↵
2 output label 3↵
...
↵
```

3.1.7 Checking the connection

While the connection to the Videohub Server is established, a client may send a special no-operation command to check that the Videohub Server is still responding:

```
PING:↵
↵
```

If the Videohub Server is responding, it will respond with an ACK message as for any other recognized command.

SECTION 3 Videohub RS-422 Protocol

3.2 Videohub RS-422 Protocol

3.2.1 General

The RS-422 protocol can be used to control Micro Videohub and Smart Videohub, as RS-422 slave devices, for router crosspoint switching.

These Videohub models implement the router (server) side of the Leitch Serial Pass-Through Protocol as specified in section 4 of Leitch document SPR-MAN revision D. The router does not support the Leitch Terminal Protocol protocol (chapter 3).

This document describes the commands and parameters in the protocol that are relevant and supported by Videohub. Other commands and parameters specified in the Leitch protocol are accepted but ignored.

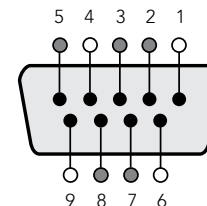
The RS-422 serial port is configured as 9600 N81.

The protocol is line-oriented, with a maximum length of 250 characters per command. Each command from the client should be terminated with a line-feed (\n). Carriage-return (\r) characters will be ignored. Each response from the Videohub Server will be terminated with a line-feed (\n).

Sources, destinations and levels are always specified in base-16, numbered from zero. Levels are always between 0 and 15 ("F"). Videohubs only have one valid level - level zero.

On connecting to the serial port, the client should send a line-feed. The Videohub Server will respond with an > character prompt, which is not followed by a line-feed. Receiving the prompt indicates that a connection has been established. The same prompt will be issued after each command received by the Videohub Server.

In the following documentation, commands in orange and values in blue must be typed literally, including any spaces. In the following example of an immediate command using destination port 7 and source port 3, @ X:0/destination,source would be entered as: @ X:0/6,2



Videohub RS-422 Router Control

Pin	Function
1	Ground
2	Tx- (Data sent by router)
3	Rx+ (Data received by router)
4	Ground
5	NC (No connect)
6	Ground
7	Tx+ (Data sent by router)
8	Rx- (Data received by router)
9	Ground

Videohub RS-422 Router Control pin connections.

SECTION 3 Videohub RS-422 Protocol

3.2.2 Notifications

Once connected, if status reporting is enabled, the client will receive a notification message when a route changes on the Videohub Server. The notifications take one of two forms:

S:0destination,source Routing change

This message indicates that the specified source port is now routed to the specified destination.

V:0destination,source Preset routing notification

This message indicates that the current preset includes a route from the specified source to the specified destination.

3.2.3 Global Commands

All pass-through commands are preceded by an @ symbol and a space.

The following client commands are supported:

@ ! disable status reporting

Status reporting is disabled by default.

@ ? enable status reporting

Status reporting is enabled.

@ Z: reset routing table

Routing is reset so that the first source is routed to all destinations.

SECTION 3 Videohub RS-422 Protocol

3.2.4 Immediate Commands

@ X:0/destination,source change route

@ X:0/destination,source/destination-2,source-2... change multiple routes

The specified source ports are routed to the specified destinations.

Any routing changes will trigger S: notifications

@ X?0destination request individual route status

The source routed to a the specified destination will be returned as an S: notification.

@ S?0 request all ports route status

Each source & destination port pair will be returned as S: notifications

3.2.5 Salvo Commands

@ P:0/destination,source queue route change

@ P:0/destination,source/destination-2,source-2... queue multiple route changes

The specified routing changes are added to the current salvo for later execution.

@ P?0destination request individual port status in salvo

If a routing change for the specified destination port is queued, the route will be returned as a V: notification.

@ V?0 request all ports status in salvo

Each queued routing change in the salvo is reported as a V: notification.

@ B:C clear salvo

@ B:R clear salvo

Any queued changes are discarded and the salvo is reset.

@ B:E execute salvo

Any queued changes are executed and each routing change will be returned as an S: notification.

